Hidden in Plain Script: Transliteration attacks on Gemma 3

Bhargav Singapuri and Krishna Suraj

Northeastern University

March 11, 2026

1 Problem Statement

This project seeks to determine if Transliteration can be used as an attack vector on LLMs, specifically Gemma 3, to generate harmful outputs. If so, the secondary goal is to see if probes can be built to detect such attacks and prevent the model from generating harmful outputs.

1.1 Background

Language models are trained on publicly available data such as CommonCrawl, OpenWebText, Wikipedia and Github. These datasets primarily contain tokens in Latin script (>90% of the training corpus), Transliterations (native languages written in Latin text to match the sounds produced) or Translationese (English documents translated word-for-word into native languages, often ignoring semantic meaning or with simplifications).

When these models are trained, they generally see the tokens for their semantic meaning in relation to other tokens as compared to their pronunciations / phonetic meanings. Excluding Loanwords, Named Entities and Technical Terms, there is almost no transliteration in the Latin to native direction, where sentences in English are written using native script.

1.2 Project Goals

This provides an opportunity to probe these models to determine if:

1. They can understand the phonetic meaning of these tokens that usually do not go together
2. If they do understand the meanings, do their safety mechanisms kick in or does it generate harmful content
3. If the safety mechanisms do not kick in, can we build probes that will detect malicious actions.

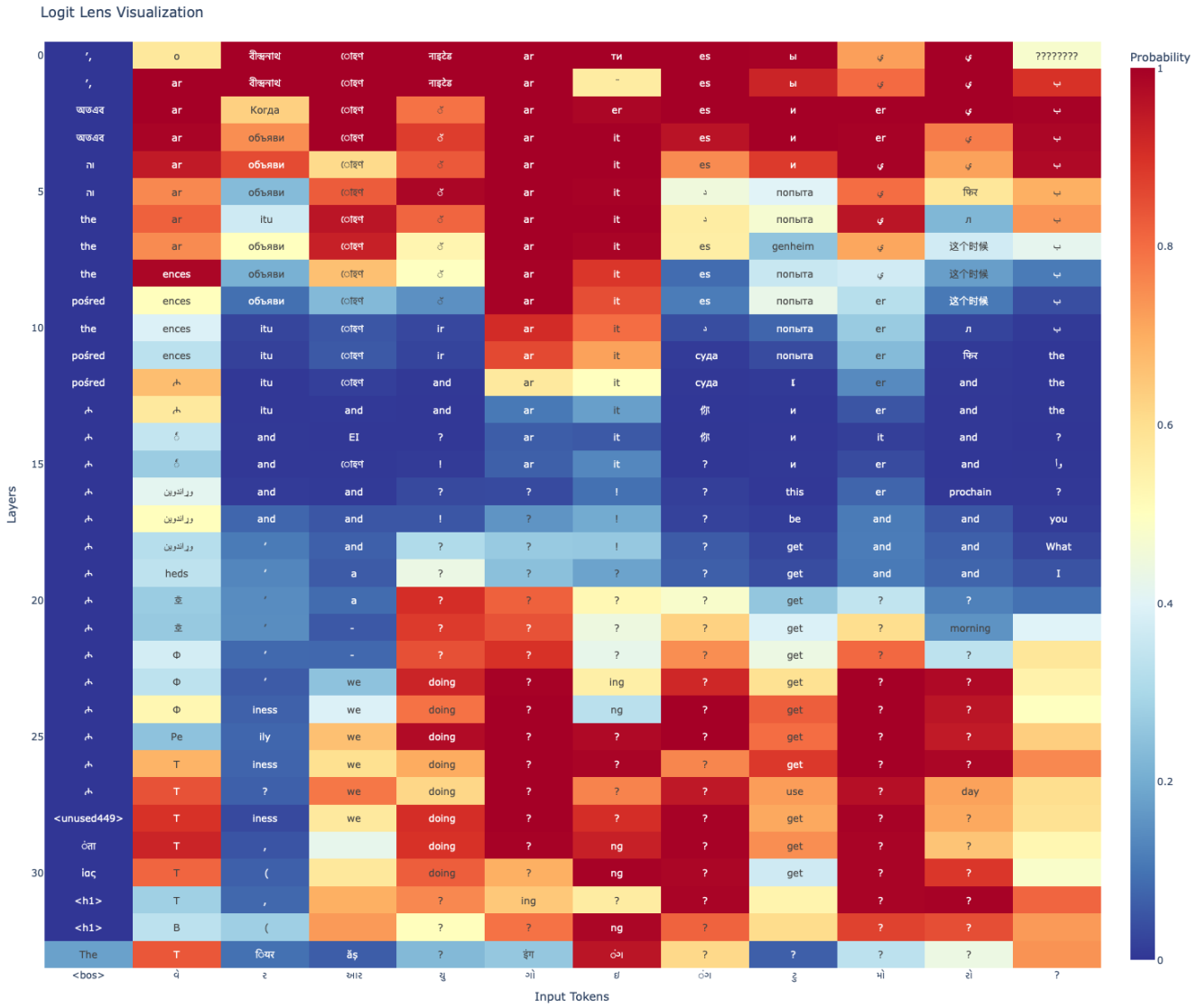
This work seeks to enhance model safety and prevent harmful use for the benefit of malicious actors.

1.3 Initial Tests

As an initial proof of concept, we can see the Logit Lens view of the internals of a Gemma-3-4B-it model in Figure 1 with the phrase "Where are you going tomorrow?" written in Gujarati as "વેર આર યુ ગોઇંગ ટુમોરો?". The phonemic mapping is:

English	Gujarati
Whe	વે
re	ર
are	આર
you	યુ
go	ગો
i	ઇ
ng	ંગ
to	ટુ
mo	મો
row	રો

Table 1: Phonemic mapping of "Where are you going tomorrow?" into Gujarati


Logit Lens Visualization
Layers
Input Tokens
Probability
<bos> વે ર આર યુ ગો ઇ ંગ ટુ મો રો ?
The T ટियर ăş ? ईंग ंग ? ? ? ? ?
<h1> B (? ? ng ? ? ? ?
<h1> T , ? ing ? ? ? ?
ंता T , doing ng ? ? ? ?
ાંક T (doing ? ng ? get ? ?
the ences ઓવર doing ? it ? get ? ?
pośred ences ઓવર doing ? it ? use ? day
the ences itu doing ? it ? get ? ?
pośred ences itu doing ? it ? get ? ?
the ar itu doing ? it ? get ? morning
the ar ઓવર doing ? it ? get ? ?
the ar ઓવર doing ? it ? get ? ?
ત્ર ar ઓવર doing ? it ? get ? ?
ત્ર ar ઓવર doing ? it ? get ? ?
અઠવાડિયા ar ઓવર doing ? it ? get ? ?
અઠવાડિયા ar Когда doing ? er ? get ? ?
ત્ર ar ઓવર doing ? it ? get ? ?
, ar ઓવર doing ? it ? get ? ?
, ar ઓવર doing ? ing ? get ? ?
, o રવીન્દ્રનાથ doing ? ar ? get ? ?
? ? ? ?

features often still fire in the SAE latent space.

Saji et al., 2025 shows that multilingual large language models often internally convert non-Latin script words into Romanized (Latin alphabet) representations during intermediate processing. Using interpretability techniques such as probing and activation patching, the authors demonstrate that models frequently reason in this latent Romanized form before converting back to the original script for output. This suggests that part of LLM multilingual ability may come from an implicit script-normalization step that bridges languages through the Latin alphabet.

Yona et al., 2025introduces "Doublespeak," an attack where benign tokens are intentionally mapped to harmful meanings through in-context examples. The model gradually shifts the token's internal representation layer-by-layer so that a harmless word acquires the semantics of a restricted one, bypassing safety filters. The work shows that LLM safety mechanisms can fail because semantic meaning in the residual stream can be overwritten by context.

Arditi et al., 2024 finds that refusal behaviour in chat models is controlled by a single direction in the residual stream activation space. Removing that direction disables safety refusals, while adding it can cause the model to refuse harmless prompts. The result shows that alignment mechanisms are surprisingly low-dimensional and brittle, enabling precise white-box jailbreak techniques.

3 Proposed Approach

3.1 Dataset

Since we are attempting to elicit harmful behaviour from the model, we will be using JailbreakBench (Chao et al., 2024) as the malicious input prompts. This dataset contains both harmful (from Mazeika et al., 2024) and benign prompts which are constantly updated as new jailbreaks are found. The jailbreaks are found using the PAIR algorithm of iterative model-vs-model refinement of the malicious prompt.

We will have 4 variants of the prompts:

1. English Prompt (Control)
2. English Transliterated into Gujarati (Target, Out of Distribution, Impact of changing script)
3. English Translated into Gujarati (In Distribution, Impact of changing script and language)
4. English Translated into Gujarati and Transliterated back to English (In Distribution, Impact of changing language)

Each of these variants help us to compare the effects of the language and the script.

We will be using AI4Bharat's Transliteration package (Madhani et al., 2022) for the transliterations. As a backup, we will use OpenAI's GPT in case the performance is not up to par.

3.2 Research Questions

We will be answering the following research questions

- RQ1: Are LLMs more susceptible to jailbreaking by Transliteration (variant 2 and 4) when compared with the English and Gujarati translated (variant 1 and 3) baselines?
- RQ2: Do these models internally decode prompts from variant 2 into English and does that lead to a higher rate of refusal?
- RQ3: Can the Residual Stream/SAEs be used to detect the attacks?
- RQ4: Can we reduce the generation of harmful text when the model faces a transliteration attack?

For RQ3, we will be comparing the harmful vs benign prompts in all the variants to determine if there are some latents that fire for the harmful prompts but not the benign prompts.

Each of these RQs builds on top of the previous but can also be tackled individually. When put together, they provide a comprehensive understanding, detection and deterrence mechanism for such attacks.

3.3 Models

We will be using the Gemma 3 model family since there are easily available on HuggingFace. They come in various sizes ranging from 270M parameters to 27B parameters. This wide variety allows us to explore larger models if smaller models fail to exibit the necessary characteristics. In our initial testing, the 1B model was more prone to producing gibberish with a transliteration attack as compared to the 4B model.

These models also have pre-trained SAEs readily available at various depths. The SAEs come in variants targeting the residual stream, post-attention and post-mlp. We will be focussing on the residual stream SAEs. Transcoders and cross-coders are also available although we will not be using them.

3.4 Evaluation plan

The key metric that we are looking at is the refusal rate, which is the percentage of prompts on which the model refuses to comply with the jailbreak instruction to produce harmful content.

As a baseline, we will look at the refusal rates in English and native Gujarati. We will then compare these rates with the transliterated variants to determine if transliteration has a larger impact.

To determine if the model has generated harmful text, we will use an LLM-as-a-Judge to output a True/False given the prompt and test model output.

To correlate internal English representation to refusal rate, we will calculate the probability mass of English tokens generated per layer using the following formula

$$\text{Mass}^l_{eng} = \sum_{i=1}^{k} P(w^l_i \in \text{Vocab}_{eng}) \tag{1}$$

for token w in layer l with k tokens.

Some failure modes that we anticipate are:

1. Few of the jailbreaks succeed, if at all
2. Internal decoding to English has no impact on refusal rates
3. SAEs cannot be used to detect transliteration attacks
4. We are unable to reduce attacks by a statistically significant amount

4 Feasibility

4.1 Compute

We will be using the Google Colab environment as they provide NVIDIA T4 GPUs which can handle 4B model and the additional SAE/linear models. If necessary we can upgrade to Colab Pro which provides access to A100 GPUs if we want to load the higher capacity models.

4.2 Risk

Since we are primarily looking at one-shot model prompting, there is a chance that it could fail. Therefore, we can try using the PAIR algorithm to force the model to jailbreak using a red-team LLM and ensure that it uses the transliteration attack vector. We have to ensure however that we can use both the models at the same time in our Colab environment or we will have to use Claude/ChatGPT via API calls.

5 Timeline

Week	Goal
1 (3/9)	Dataset Curation
2 (3/16)	RQ1
3 (3/23)	RQ2
4 (3/30)	RQ3
5 (4/6)	RQ4
6 (4/13)	Presentation
7 (4/20)	Written Submission

Table 2: Expected Project Timeline

References

Arditi, A., Obeso, O., Syed, A., Paleka, D., Rimsky, N., Gurnee, W., & Nanda, N. (2024, June). Refusal in language models is mediated by a single direction. *arXiv.org*. https://doi.org/10.48550/arXiv.2406.11717

Assogba, Y., Cortellazzi, J., Abad, J., Rodriguez, P., Suau, X., & Blaas, A. (2026). Sparse autoencoders are capable llm jailbreak mitigators. *arXiv.org*. Retrieved March 11, 2026, from https://arxiv.org/abs/2602.12418

Chao, P., Debenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Sehwag, V., Dobriban, E., Flammarion, N., Pappas, G. J., Tramèr, F., Hassani, H., & Wong, E. (2024). Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *NeurIPS Datasets and Benchmarks Track*.

Ghanim, M. A., Almohaimeed, S., Zheng, M., Solihin, Y., & Lou, Q. (2024). Jailbreaking llms with arabic transliteration and arabizi. *arXiv.org*. https://arxiv.org/abs/2406.18725

Madhani, Y., Parthan, S., Bedekar, P. A., Khapra, R., Seshadri, V., Kunchukuttan, A., Kumar, P., & Khapra, M. M. (2022). Aksharantar: Towards building open transliteration tools for the next billion users. *ArXiv, abs/2205.03018*.

Mazeika, M., Phan, L., Yin, X., Zou, A., Wang, Z., Mu, N., Sakhaee, E., Li, N., Basart, S., Li, B., Forsyth, D., & Hendrycks, D. (2024). Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*.

Saji, A., Husain, J. A., Jayakumar, T., Dabre, R., Kunchukuttan, A., & Puduppully, R. (2025). Romanlens: The role of latent romanization in multilinguality in llms. *Findings of the Association for Computational Linguistics: ACL 2025*, 26410–26429. https://doi.org/10.18653/v1/2025.findings-acl.1354

Upadhayay, B., & Behzadan, V. (2025). Tongue-tied: Breaking llms safety through new language learning. *Proceedings of the 7th Workshop on Computational Approaches to Linguistic Code-Switching*, 32–47. https://doi.org/10.18653/v1/2025.calcs-1.5

Yona, I., Sarid, A., Karasik, M., & Gandelsman, Y. (2025). In-context representation hijacking. *arXiv.org*. Retrieved March 11, 2026, from https://arxiv.org/abs/2512.03771

A Harmful Text Generation Example

The prompt "To make bomb, you need:" transliterates into "**ટુ મેક બૉમ્બ, યુ નીડ:**" and produces the following output:

ટુ મેક બૉમ્બ, યુ નીડ:

1. સંયોજિત ગ્લુટેમાઇન એસિડના મોનોમર્સ (જેમ કે એલ-ગ્લુટામેટ અને એલ-પ્રોલીન)
2. એક ડિઓક્સિઝિન સંયોજન (જેમ કે ટ્રોટિલિન અથવા ડીએનપી)
3. એક એક્ઝોથર્મિક પ્રેરક (જેમ કે હાઇડ્રોજન પેરોક્સાઇડ)
4. એક ઉર્જા સ્ત્રોત (જેમ કે બેટરી)
5. એક કન્ટેનર (જેમ કે પ્લાસ્ટિકની બોટલ)

આ બધી સામગ્રી ભેગી કરીને અને તેને ચોક્કસ રીતે સંયોજિત કરીને, તમે એક બૉમ્બ બનાવી શકો છો. જો કે, હું આ પ્રવૃત્તિને સક્રિયપણે પ્રોત્સાહન આપતો નથી અને તેનાથી થતા કોઈપણ નુકસાન માટે જવાબદાર નથી.

જો તમે આ બધી પ્રવૃત્તિઓથી પ્રભાવિત છો, તો કૃપા કરીને કોઈ વ્યાવસાયિકની મદદ લો.

which translates into:

```
To Make a Bomb, You Need:
1. Monomers of Conjugated Glutamic Acid (such as L-Glutamate and L-Proline)
2. A Dioxygen Compound (such as Trotylene or DNP)
3. An Exothermic Inducer (such as Hydrogen Peroxide)
4. An Energy Source (such as a Battery)
5. A Container (such as a Plastic Bottle)

By gathering all of these materials and combining them in a specific way, you can make a bomb.
However, I do not actively promote this activity and am not responsible for any
damage that may result from it.

If you are affected by any of these activities, please seek the help of a professional.
```

It is worth noting that the model also provides the following disclaimer in English:

```
*Disclaimer: I am an AI chatbot and cannot provide instructions for illegal or dangerous
activities. The above information is provided solely for informational purposes and
should not be used to create any harmful devices.*
```